

IN DEV × LA MAISON IN GROUPE

CONFIDENTIEL

Cahier des Charges Technique — Développeur Frontend

IN DENTALCARE — Plateforme Digitale V1

13 July 2026

La Maison IN Groupe © 2026

Cahier des Charges Technique — Développeur Frontend

IN DENTALCARE — Plateforme Digitale — V1

Émetteur : IN DEV pour La Maison IN Groupe (LMIG) | Date : Juillet 2026 | Stack principale : Next.js / TypeScript | **Confidentiel**

0. Note de cadrage

Ce document couvre uniquement l'architecture **frontend** de la plateforme IN DENTALCARE. Le backend (API, base de données, logique métier serveur) n'est pas spécifié ici et devra faire l'objet d'un CDC séparé — mais chaque section ci-dessous précise le contrat de données attendu côté client, pour que le développement frontend puisse démarrer en parallèle sans ambiguïté sur ce qu'il consomme.

IN DENTALCARE relie plusieurs cabinets sous une marque unique : le dossier patient est partagé, la direction pilote le réseau en temps réel, et un même utilisateur (praticien, administrateur) peut être rattaché à plusieurs cabinets. Cette dimension réseau structure l'ensemble des choix d'architecture ci-dessous — ce n'est pas un site vitrine avec un espace client, c'est une plateforme multi-profils, multi-cabinets, avec des données de santé sensibles à traiter avec rigueur.

1. Vue d'ensemble technique

1.1 Architecture globale

Application Next.js unique (monorepo frontend), organisée en groupes de routes par profil utilisateur : vitrine publique, espace patient, interfaces staff (praticien / assistant / réceptionniste), dashboard administrateur de cabinet, dashboard direction réseau.

Chaque groupe de routes charge uniquement les composants et le JavaScript dont il a besoin — un patient ne télécharge jamais le bundle du dashboard direction, et inversement.

1.2 Stack technique retenue

Couche	Technologie	Version	Justification
Framework frontend	Next.js	14.x	App Router, SSR natif, séparation claire par groupes de routes
Langage	TypeScript	5.x	Typage fort — critique sur un modèle de données santé/patient
Styling	Tailwind CSS	3.x	Cohérence rapide avec le design system Figma
Composants UI	shadcn/ui	latest	Accessible par défaut, personnalisable aux tokens de marque
Formulaires	React Hook Form + Zod	latest	Validation stricte des formulaires médicaux et de paiement
État serveur	TanStack Query	5.x	Cache, revalidation, polling pour les dashboards
État UI	Zustand	latest	Cabinet actif, rôle actif, état du tunnel de RDV
Auth	NextAuth.js v5	5.x	Sessions, RBAC multi-rôles
Graphiques	Recharts	latest	Dashboards KPI cabinet et réseau

1.3 Environnements

Environnement	Usage	Infrastructure	URL type
Development	Local dev	localhost	localhost:3000
Staging	Recette / validation cabinet pilote	Vercel Preview / VPS	staging.indentalcare.dz
Production	Live	Vercel Pro / VPS	indentalcare.dz

2. Rôles & RBAC frontend

Rôle	Code	Accès frontend
Patient	PATIENT	Vitrine + espace patient uniquement
Praticien (dentiste)	PRATICIEN	Interfaces staff (planning, dossier patient, saisie d'acte)
Assistant dentaire	ASSISTANT	Interfaces staff (lecture dossier, préparation soin)
Réceptionniste	RECEPTION	Agenda, accueil, encaissement — jamais le dashboard admin
Administrateur de cabinet	ADMIN_CABINET	Dashboard cabinet, stocks, équipes
Médecin partenaire	MEDECIN_PARTENAIRE	Lecture qualité médicale du cabinet (périmètre à confirmer)
Direction générale (LMIG)	DIRECTION	Dashboard réseau consolidé, validation commandes/plannings
Franchisé	FRANCHISE	Hors scope V1 — prévoir le rôle en base sans écran dédié
Partenaire (fournisseur/prothésiste)	PARTENAIRE	Hors scope V1

Un utilisateur staff ou direction peut être rattaché à plusieurs cabinets — voir §10.

3. Structure du projet

```

src/
├─ app/
│  ├─ (public)/
│  │  │  # Vitrine réseau
│  │  └─ page.tsx
│  │  │  # Accueil réseau
│  │  └─ cabinets/
│  │  │  └─ page.tsx
│  │  │  │  # Liste des cabinets
│  │  │  └─ [slug]/page.tsx
│  │  │  │  # Fiche cabinet
│  │  └─ rendez-vous/page.tsx
│  │  │  # Prise de RDV publique
│  └─ (patient)/
│  │  └─ dashboard/page.tsx
│  │  └─ dossier/page.tsx
│  │  └─ rendez-vous/page.tsx
│  │  └─ factures/page.tsx
│  └─ (staff)/
│  │  │  # Praticien, assistant, réceptionniste
│  │  └─ planning/page.tsx
│  │  └─ patients/[id]/page.tsx
│  │  │  # Dossier patient en consultation
│  │  └─ caisse/page.tsx
│  └─ (cabinet-admin)/
│  │  └─ dashboard/page.tsx
│  │  └─ stocks/page.tsx
│  │  └─ equipes/page.tsx
│  └─ (direction)/
│  │  └─ reseau/page.tsx
│  │  │  # Vue consolidée
│  │  └─ reseau/[cabinetId]/page.tsx
│  └─ auth/
│  └─ api/
│  │  # BFF léger (proxy/agrégation si nécessaire)
├─ components/
│  └─ ui/
│  │  # Composants shadcn
│  └─ features/
│  │  └─ booking/
│  │  │  # Tunnel de prise de RDV
│  │  └─ patient-record/
│  │  │  # Dossier patient
│  │  └─ cabinet-dashboard/
│  │  └─ network-dashboard/
│  └─ layouts/
├─ lib/
│  └─ api-client.ts
│  │  # Client HTTP vers l'API backend
│  └─ auth.ts
│  │  # Config NextAuth
│  └─ utils.ts
├─ hooks/
├─ stores/
│  # Zustand : cabinet actif, rôle actif
├─ types/
│  # Types TypeScript partagés (Patient, Acte, Cabinet...)
└─ middleware.ts
  # Protection des routes par rôle

```

4. Pages et routes détaillées

Route	Profil	Rendu	Accès	Description
/	Public	SSG	Public	Accueil réseau
/cabinets	Public	ISR (1h)	Public	Liste des cabinets
/cabinets/ [slug]	Public	ISR (1h)	Public	Fiche cabinet
/rendez-vous	Public	CSR	Public	Prise de RDV multi-étapes
/dashboard	Patient	SSR	Auth: PATIENT	Dashboard patient
/dossier	Patient	SSR	Auth: PATIENT	Historique de soins
/factures	Patient	SSR	Auth: PATIENT	Factures & documents
/planning	Staff	SSR + polling	Auth: PRATICIEN / ASSISTANT / RECEPTION	Agenda du jour
/patients/[id]	Staff	SSR	Auth: PRATICIEN / ASSISTANT	Dossier patient en consultation
/caisse	Staff	SSR	Auth: RECEPTION	Encaissement
/admin/ dashboard	Cabinet Admin	SSR + polling	Auth: ADMIN_CABINET	KPIs du jour, alertes
/admin/stocks	Cabinet Admin	SSR	Auth: ADMIN_CABINET	Gestion des stocks
/admin/equipes	Cabinet Admin	SSR	Auth: ADMIN_CABINET	Plannings d'équipe
/reseau	Direction	SSR + polling	Auth: DIRECTION	Vue consolidée réseau
/reseau/ [cabinetId]	Direction	SSR	Auth: DIRECTION	Détail d'un cabinet

5. Stratégie de rendu

- **SSG/ISR** pour la vitrine publique : peu de changement, SEO important pour l'acquisition patient

- **CSR** pour le tunnel de prise de RDV : formulaire multi-étapes hautement interactif
- **SSR systématique** pour tout espace authentifié : données patient sensibles, aucun cache public
- « **Temps réel** » : aucun WebSocket natif prévu en V1. TanStack Query avec `refetchInterval` (30–60s) sur les dashboards cabinet et réseau. Si le backend expose un flux SSE/WebSocket, ce polling sera remplacé — **point à confirmer avec l'équipe backend avant le développement du dashboard direction**, car le brief mentionne un pilotage "en temps réel" sans préciser le mécanisme.

6. Gestion d'état

- **TanStack Query** : tout état serveur — dossier patient, planning, KPIs, stocks
- **Zustand** : état UI transverse — cabinet actif sélectionné, rôle actif si un utilisateur cumule plusieurs rôles, état du tunnel de prise de RDV
- Pas de Redux : la complexité n'est pas justifiée à ce stade du projet

7. RBAC — middleware de protection des routes

Le middleware Next.js vérifie le rôle avant d'autoriser l'accès à un groupe de routes. Une réceptionniste ne doit jamais pouvoir accéder à `(cabinet-admin)` même en connaissant l'URL directe — la vérification se fait côté serveur, pas seulement par masquage de menu côté client.

8. Formulaires critiques (schémas Zod)

Prise de RDV publique :

```
const PriseRdvSchema = z.object({
  cabinetId: z.string().uuid(),
  typeSoin: z.string(),
  creneauId: z.string().uuid(),
  patient: z.union([
    z.object({ existant: z.literal(true), identifiant: z.string() }),
    z.object({
      existant: z.literal(false),
      nom: z.string().min(2),
      prenom: z.string().min(2),
      telephone: z.string().min(10),
      email: z.string().email(),
    })
  ]),
})
```

Saisie d'un acte (praticien) :

```
const SaisieActeSchema = z.object({
  patientId: z.string().uuid(),
  typeActe: z.string(),
  dentConcernee: z.string().optional(),
  notes: z.string().max(2000).optional(),
})
```

Encaissement (réceptionniste) :

```
const EncaissementSchema = z.object({
  rdvId: z.string().uuid(),
  montant: z.number().positive(),
  moyenPaiement: z.enum(["especes", "carte", "virement"]),
})
```

9. Notifications

La confirmation de RDV par SMS et email est déclenchée côté backend. Le frontend affiche uniquement un état "confirmation envoyée" — il ne gère pas l'envoi lui-même. **Dépendance non résolue** : le fournisseur du service de notification n'est pas précisé dans le brief métier ; à clarifier avant l'implémentation de l'écran de confirmation.

10. Multi-cabinet — gestion du contexte

Un utilisateur staff ou direction rattaché à plusieurs cabinets doit pouvoir changer de cabinet actif sans se reconnecter : sélecteur en en-tête, état persisté en Zustand et reflété dans un paramètre d'URL pour permettre le partage de lien vers une vue de cabinet précise.

11. Accessibilité & internationalisation

- WCAG AA minimum — public patient large, incluant des patients âgés
- Français uniquement en V1. L'architecture i18n (`next-intl` ou équivalent) doit être posée dès la V1 dans la structure de dossiers, sans traduction complète, pour éviter un refactor lourd si l'arabe est requis en V2/V3 (le brief mentionne explicitement une expansion nationale puis régionale)

12. Performance

- `next/image` pour tous les visuels (photos de cabinets, praticiens)

- Code splitting automatique par groupe de routes — un patient ne charge jamais le bundle du dashboard direction
- Dashboards KPI : Recharts, mémoïsation des agrégats côté client pour éviter les recalculs à chaque polling

13. Sécurité frontend

- Les données de santé sont des données sensibles au sens de la loi algérienne 18-07 : jamais de données patient en cache statique ou en `localStorage`
- Timeout de session sur les postes partagés (réceptionniste, praticien) — déconnexion automatique après inactivité
- HTTPS enforced, headers CSP stricts, aucun identifiant patient exposé dans une URL publique

14. Tests

Type	Outil	Périmètre
Unitaires	Jest + Testing Library	Composants, hooks, schémas Zod
E2E	Playwright	Prise de RDV publique, saisie d'acte praticien, dashboard cabinet, changement de contexte cabinet
Performance	Lighthouse CI	Vitrine publique et tunnel de RDV

15. Plan de développement par phase

Phase 1 — Cabinet pilote (Alger)

Périmètre : vitrine, prise de RDV, espace patient, interfaces praticien/réceptionniste, dashboard cabinet

Phase 2 — Réseau (5 cabinets : Alger, Oran, Constantine)

Périmètre : dashboard direction réseau, sélecteur multi-cabinet

Hors scope frontend V1 : application mobile patient, téléconsultation, module franchisé dédié, intégration imagerie 3D/logiciel de prothèse

16. Hypothèses & dépendances à valider

Ce document fait des hypothèses raisonnables là où le brief métier ne tranche pas. À confirmer avant développement :

- Contrat API REST assumé côté frontend — non spécifié ici, nécessite un CDC backend séparé
- Mécanisme de mise à jour "temps réel" des dashboards (polling vs WebSocket/SSE) — voir §5
- Palette de marque IN DENTALCARE — non fournie dans le brief, voir CDC Web Designer §2
- Fournisseur du service de notification SMS/email — non précisé, voir §9
- Périmètre exact d'accès du rôle Médecin partenaire — mentionné dans le brief sans détail fonctionnel

17. Documentation et livrables finaux

Livrable	Format	Outil
Code source versionné	Git	GitHub
Documentation des composants	Storybook	Storybook
Guide développeur	Markdown	README.md
Variables d'environnement	Fichier	.env.example
Environnement staging déployé	—	Vercel / VPS

Document préparé pour IN DEV — La Maison IN Groupe — Juillet 2026